LexAI: The Complete Frontend Learning Guide & Viva Preparation

Welcome to your complete LexAI Frontend Master Guide. This document contains everything you need to understand your UI, HTML, CSS, JavaScript, and how to defend your frontend choices during your MCA Viva.

MODULE 1 & 2: Frontend Structure & HTML Explained

Your frontend relies on **Server-Side Rendering (SSR)**. Flask generates the HTML using **Jinja2** before sending it to the browser.

The Structure

- `templates/` : Contains all HTML files (`login.html` , `dashboard.html`).
- `static/css/style.css` : Your custom visual rules.
- `static/js/chatbot.js` : The JavaScript that powers the chat interface without refreshing the page.

`base.html` Explained Line-by-Line

`base.html` is your "master layout". Other pages slide into its `{% block content %}`.

Code / Line	Simple Meaning	Why It Is Used
`<!DOCTYPE html>`	Tells the browser this is an HTML5 document.	Prevents rendering bugs in older browsers.
`<link href="...bootstrap.min.css">`	Loads Bootstrap 5.	Gives you instant responsive designs, grids, and buttons.
`{{ url_for('static', filename='css/style.css') }}`	Jinja2 syntax to find your CSS file.	Flask dynamically calculates the correct URL path for your CSS.
`{% if current_user.is_authenticated %}`	Jinja2 conditional.	Only shows the sidebar to users who are actively logged in.
`{% block content %}{% endblock %}`	The placeholder.	When `login.html` is loaded, its form is

Code / Line	Simple Meaning	Why It Is Used
		injected exactly here.

Explain Like I'm 5: HTML is the wooden frame of a house. Jinja2 is a smart robot builder. Before the house is shipped to the user, the robot looks at the blueprint (`{{ user.name }}`) and paints the user's actual name onto the wall.

MODULE 3: CSS & Bootstrap (Like I Am Five)

CSS is the paint and interior design. Instead of writing 10,000 lines of custom CSS, you used **Bootstrap 5**.

- **CSS Selectors:** `.btn-primary` (targets classes), `#sidebar` (targets unique IDs).
- **Margin vs Padding:** Margin is the space *outside* the box (pushing other things away). Padding is the space *inside* the box (pushing the text away from the borders).
- **Flexbox (`d-flex`):** Bootstrap uses flexbox to easily align items side-by-side.
- **Responsive Design:** Bootstrap uses grid classes like `col-md-6` . This means "take up half the screen on laptops (`md`), but take up the full screen on mobile phones."

MODULE 4 & 5: JavaScript & Forms

Your `login.html` and `register.html` use standard HTML `<form>` tags.

Code / Line	Simple Meaning	Why It Is Used
`<form method="POST" action="/login">`	Sends secure data to the `/login` route.	POST hides the password. GET would put the password in the URL!
`<input type="email" name="email" required>`	The email box.	`name="email"` is critical. Flask reads this exact name via `request.form['email']`.

JavaScript (`chatbot.js`): Normally, submitting a form refreshes the whole page. Your Chat interface cannot refresh the page, otherwise, you'd lose the chat history! We use JavaScript to intercept the "Send" button click, and we use the `fetch()` API to send the message to Flask in the background (AJAX).

MODULE 6 & 7: The Complete Frontend-to-Backend Connection

Trace: Uploading a Document 1. **User Interaction:** User clicks `<input type="file" name="document">` and hits Submit. 2. **Request:** The browser packages the file as `multipart/form-data` and sends an HTTP POST request. 3. **Backend Routing:** Flask catches this at `@document_bp.route('/upload', methods=['POST'])`. 4. **Processing:** Flask saves the file, and PyMuPDF extracts the text. 5. **Response:** Flask renders the `dashboard.html` template. 6. **Interface**

Update: The browser clears the upload screen and shows the new dashboard with the uploaded document listed in a table.

MODULE 8 & 9: Chat Interface & AI Display

Your `chatbot.html` and `chatbot.js` handle the AI interaction. * **Markdown Rendering:** Gemini returns text with `**bold**` asterisks (Markdown). Browsers don't understand Markdown. You use `marked.js` to convert it to HTML (so it shows up as bold text). * **Sanitization:** Hackers could send malicious JavaScript in a chat message. You use `DOMPurify.sanitize()` to strip out viruses before putting the AI's message on the screen.

MODULE 10: Run & Debug Frontend on Windows

How to Debug: Press **F12** in Chrome/Edge to open Developer Tools. * **Elements Tab:** Shows the live HTML/CSS. If a button is the wrong color, click it here to see which CSS rule is overriding it. * **Console Tab:** Shows JavaScript errors. If the chat stops working, look for red text here. * **Network Tab:** Shows every API request. If an upload fails, check here to see if Flask returned a `500 Internal Server Error` or a `404 Not Found`.

How to Run on Windows: 1. Open terminal in the `LexAI` folder. 2. Run `venv\Scripts\activate`. 3. Run `python app.py`. 4. Open Chrome and go to `http://127.0.0.1:5000`.

MODULE 11: 40 MCA VIVA QUESTIONS

(FRONTEND)

1. **Did you use React or Angular for the frontend?** No, I used Server-Side Rendering (SSR) with Flask, Jinja2, and Vanilla JavaScript.
2. **What is Jinja2?** A templating engine for Python. It allows us to embed Python-like logic (`{% if %}` , `{{ variable }}`) directly into HTML files.
3. **How is `base.html` used?** It acts as the master layout containing the sidebar and CSS links. Other pages inherit it using `{% extends 'base.html' %}` to avoid duplicate code.
4. **What is Bootstrap 5?** A CSS framework that provides pre-built styling and a responsive grid system, allowing the app to look good on mobile and desktop without writing heavy custom CSS.
5. **What is the difference between an ID and a Class in CSS?** An ID (`#sidebar`) must be unique and is used only once per page. A class (`.btn-primary`) can be reused on multiple elements.
6. **How does your login form communicate with the backend?** The HTML `<form>` uses the `POST` method. The `name` attributes on the input fields are read by Flask using `request.form` .
7. **Why do we use POST for login instead of GET?** GET requests append data to the URL, which would expose the user's password in the browser history. POST sends data securely in the request body.
8. **What does the `required` attribute do in HTML?** It provides basic frontend validation, preventing the browser from submitting the form if the field is empty.
9. **Is frontend validation enough for security?** No. Frontend validation is for user experience. Hackers can bypass it, so we must always re-validate data in the Flask backend.
10. **How does the chat interface work without refreshing the page?** It uses JavaScript and the `fetch()` API (AJAX) to send messages to the server and retrieve responses asynchronously in the background.
11. **What is DOM manipulation?** Using JavaScript to dynamically change the HTML structure, like appending a new chat bubble to the screen when the AI responds.

12. **What format does the frontend use to send chat data to Flask?** It uses JSON (JavaScript Object Notation), which is a lightweight data-interchange format.
13. **How do you convert the AI's Markdown text into readable HTML?** We use the `marked.js` library on the frontend.
14. **What is Cross-Site Scripting (XSS)?** An attack where a hacker injects malicious JavaScript into the webpage.
15. **How did you prevent XSS in the chat interface?** By running all incoming text through `DOMPurify.sanitize()` before injecting it into the HTML via `innerHTML`.
16. **How do you center a `div` using Bootstrap?** By applying flexbox classes like `d-flex justify-content-center align-items-center`.
17. **What is `request.files` in Flask?** It is the object Flask uses to receive physical files (like PDFs) uploaded via a frontend `<input type="file">`.
18. **Why does the file upload form need `enctype="multipart/form-data"`?** Standard forms only send text. This attribute tells the browser to divide the file into binary parts for transmission.
19. **What does HTTP Status 404 mean?** Not Found. The user tried to visit a URL or route that does not exist.
20. **What does HTTP Status 500 mean?** Internal Server Error. The frontend sent the request, but the Python backend crashed while processing it.
21. **How do you debug CSS layout issues?** By pressing F12, opening the Elements tab, and toggling CSS rules on and off in real-time.
22. **What is `url_for()` in your HTML templates?** A Jinja2 function that dynamically generates the correct URL for a route or static file, preventing broken links if URLs change later.
23. **How does the sidebar know to hide itself on mobile phones?** Bootstrap uses media queries. Classes like `d-none d-md-block` hide the element on small screens and display it on medium (`md`) screens and larger.

24. **What is an API Endpoint?** A specific URL on the backend designed to receive requests and return data (usually JSON) rather than a full HTML page.
25. **What is a Promise in JavaScript?** A mechanism used by `fetch()` to handle asynchronous operations. It says "I promise to give you the data when the server replies, but continue running other code in the meantime."
26. **What is the difference between padding and margin?** Padding is the space inside the element's border. Margin is the space outside the border separating it from other elements.
27. **What does `{% block content %}` do?** It acts as a placeholder in `base.html`. Child templates inject their specific HTML into this block.
28. **How does the frontend know if a user is logged in?** Flask passes the `current_user` object to Jinja2. We can use `{% if current_user.is_authenticated %}` to show or hide the logout button.
29. **What is a CSS media query?** A rule in CSS that applies different styles depending on the screen size (e.g., changing font size on mobile).
30. **What is the DOM?** Document Object Model. It is the tree-like structure the browser builds from the HTML, which JavaScript can interact with.
31. **Why do we use `event.preventDefault()` in JavaScript forms?** To stop the browser's default behavior of refreshing the entire page when a form is submitted.
32. **What is the purpose of Flash messages in Flask?** To send a temporary success or error message (like "Upload Successful") from the backend to the frontend for the user to see.
33. **What is a CSRF token?** Cross-Site Request Forgery token. A hidden security string in forms that prevents malicious websites from forging requests on behalf of the user. (Note: Mention if implemented in Flask-WTF).
34. **How do loading spinners work in the chat?** When the "Send" button is clicked, JavaScript unhides the spinner HTML. When the `fetch()` response arrives, JavaScript hides the spinner again.

35. **What is the difference between `let` and `const` in JavaScript?** `const` is for variables that never change their value. `let` is for variables that can be updated.
36. **What happens if JavaScript is disabled in the browser?** The main website pages (Dashboard, Upload) will still work because they use SSR, but the real-time AI Chatbot will break because it relies on JS.
37. **What are Data Attributes (`data-*`) in HTML?** They allow us to store custom data directly in HTML tags so JavaScript can read it later (e.g., `data-document-id="12"`).
38. **How are font icons implemented?** We use Bootstrap Icons or FontAwesome, which load scalable vector icons via CSS classes like `bi bi-person`.
39. **How do you trace a slow response in the frontend?** Open F12 Developer Tools, go to the Network tab, trigger the action, and inspect the "Timing" section of the request.
40. **If you had 3 more months, what frontend framework would you switch to?** I would migrate the frontend to React or Next.js to create a true Single Page Application (SPA) for smoother, app-like transitions between pages.